
CIS437 - Cloud Computing

– Designing Cloud Applications –

Instructor: Yong Zhuang

yong.zhuang@gvsu.edu

Last Class

- Recovering from a broken SSH configuration - Serial console
- App Engine as a managed application platform
- Deploying and updating a Python/Flask app
- Using Cloud Storage with an application
- Permissions and public access

Agenda

- What makes an application “cloud-ready”?
- APIs and cloud services
- Regions, zones, and fault tolerance
- Migration and monitoring
- Scalability and elasticity
- Autoscaling with Compute Engine
- Practice / scenario

What makes a cloud application different from a normal application?



What Should a Cloud Application Handle?



Why Design for the Cloud?

...remember the advantages of cloud computing?

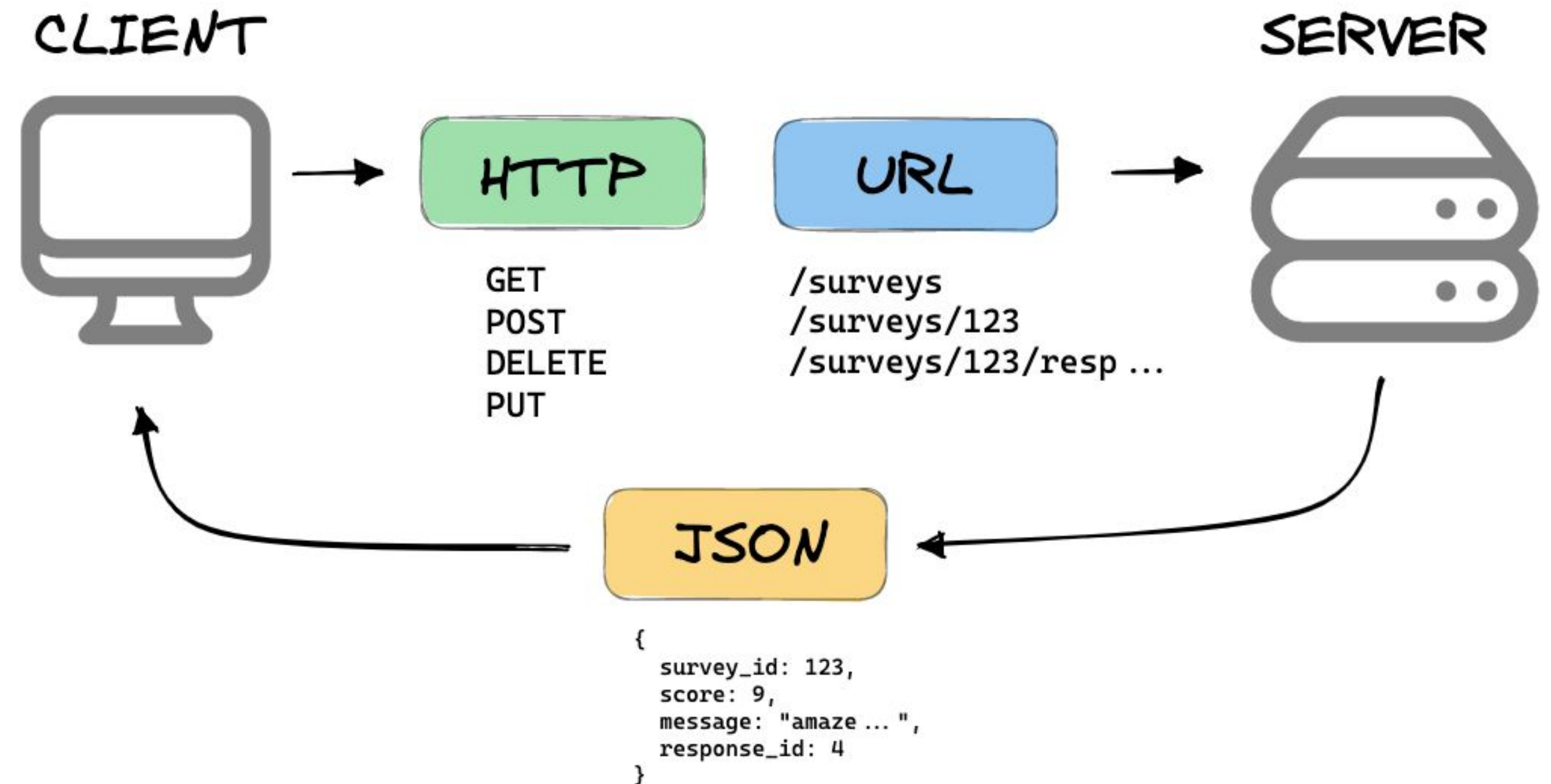
- High availability
- Fault tolerance
- Scalability
- Elasticity
- Global reach / networking
- Computing at scale

Key Design Considerations for Cloud Applications

- APIs
 - Regions and zones
 - Fault tolerance
 - Migration
 - Monitoring
 - Scalability
 - Elasticity
 - Architecture
- How does our application interact with cloud services?
 - Where should resources run?
 - What happens when something fails?
 - How do we monitor the system?
 - What happens when demand increases or decreases?

i.e., important things to consider in your cloud app

WHAT IS A REST API?



mannhowie.com

What is an API?

- An **Application Programming Interface**
- A defined way for software to communicate with another service
- Allows applications and tools to request data or actions programmatically


```
$ aws ec2 run-instances \
  --image-id ami-1a2b3c4d \
  --count 1 \
  --instance-type c3.large \
  --key-name MyKeyPair \
  --security-groups MySecurityGroup
```

(a)

```
$ openstack server create --flavor 1 --image 397e713c-b95b-4186-ad46-6126863ea0a9 \
  --security-group default --key-name KeyPair01 --user-data cloudinit.file \
  myCirrosServer|
```

(b)

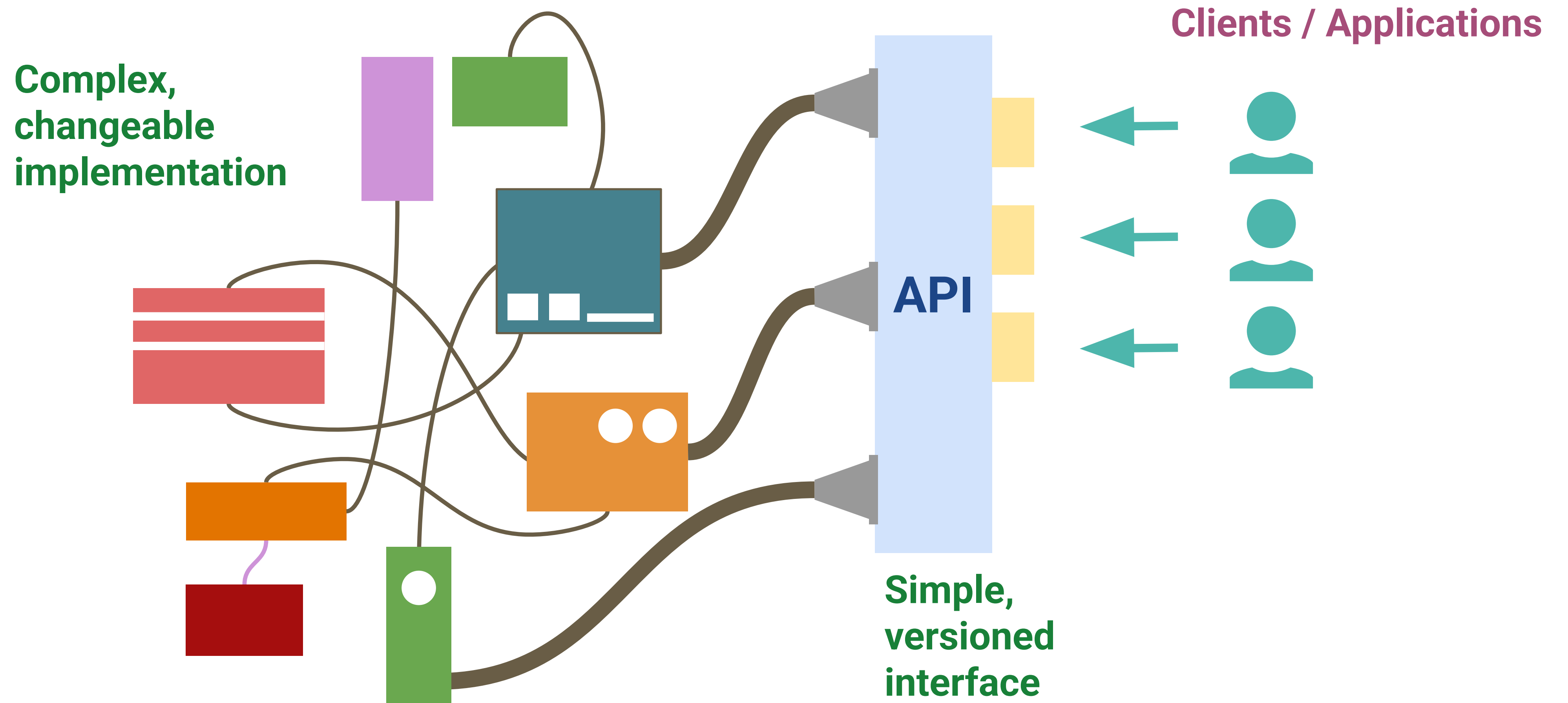
```
$ gcloud compute instances create "my-new-instance" \
  --zone="us-west1-b" \
  --image-family="tf-latest-cu92" \
  --image-project=deeplearning-platform-release \
  --maintenance-policy=TERMINATE \
  --accelerator="type=nvidia-tesla-v100,count=8" \
  --machine-type="n1-standard-8" \
  --boot-disk-size=120GB \
  --metadata="install-nvidia-driver=True"
```

(c)

Why are APIs important in the cloud?

- Cloud services expose functionality through APIs
- APIs make automation possible
- The Console and **gcloud** often use the same underlying cloud APIs

Figure 3.1: How to run a new VM by using the command-line client software provided by (a) AWS, (b) OpenStack and (c) GCP respectively.



Regions and Zones

Region: a geographic area where a cloud provider operates

- Example: **us-east1**

Zone: an isolated deployment area within a region

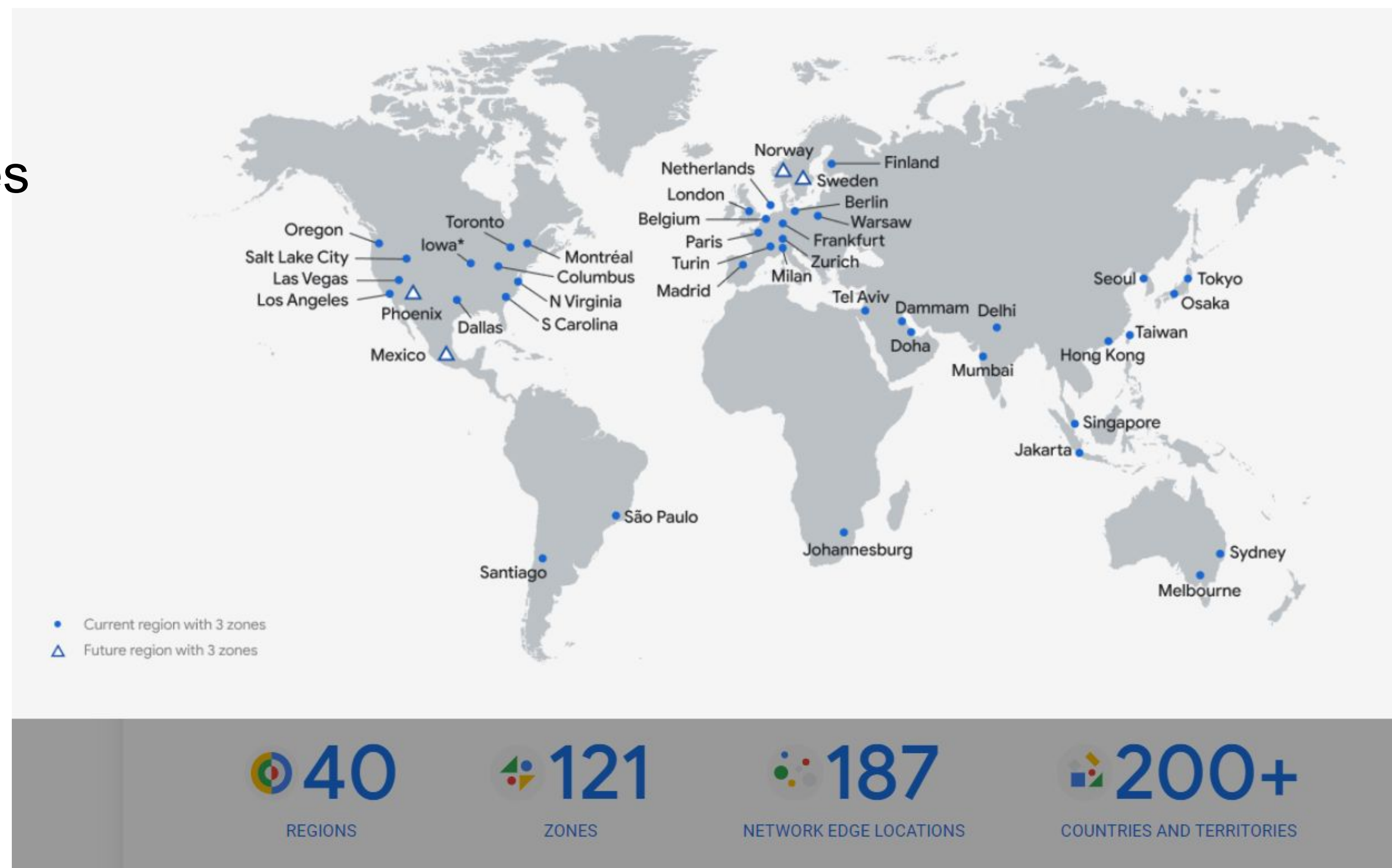
- Example: **us-east1-b**

A region usually contains multiple zones

Choose locations based on:

- latency to users
- availability / fault tolerance
- service availability
- cost and data-location requirements

Region: us-east1
├── us-east1-b
├── us-east1-c
└── us-east1-d



Cloud locations
AWS Global Infrastructure

Fault Tolerance: Designing for Failure

Failures are expected in distributed/cloud systems

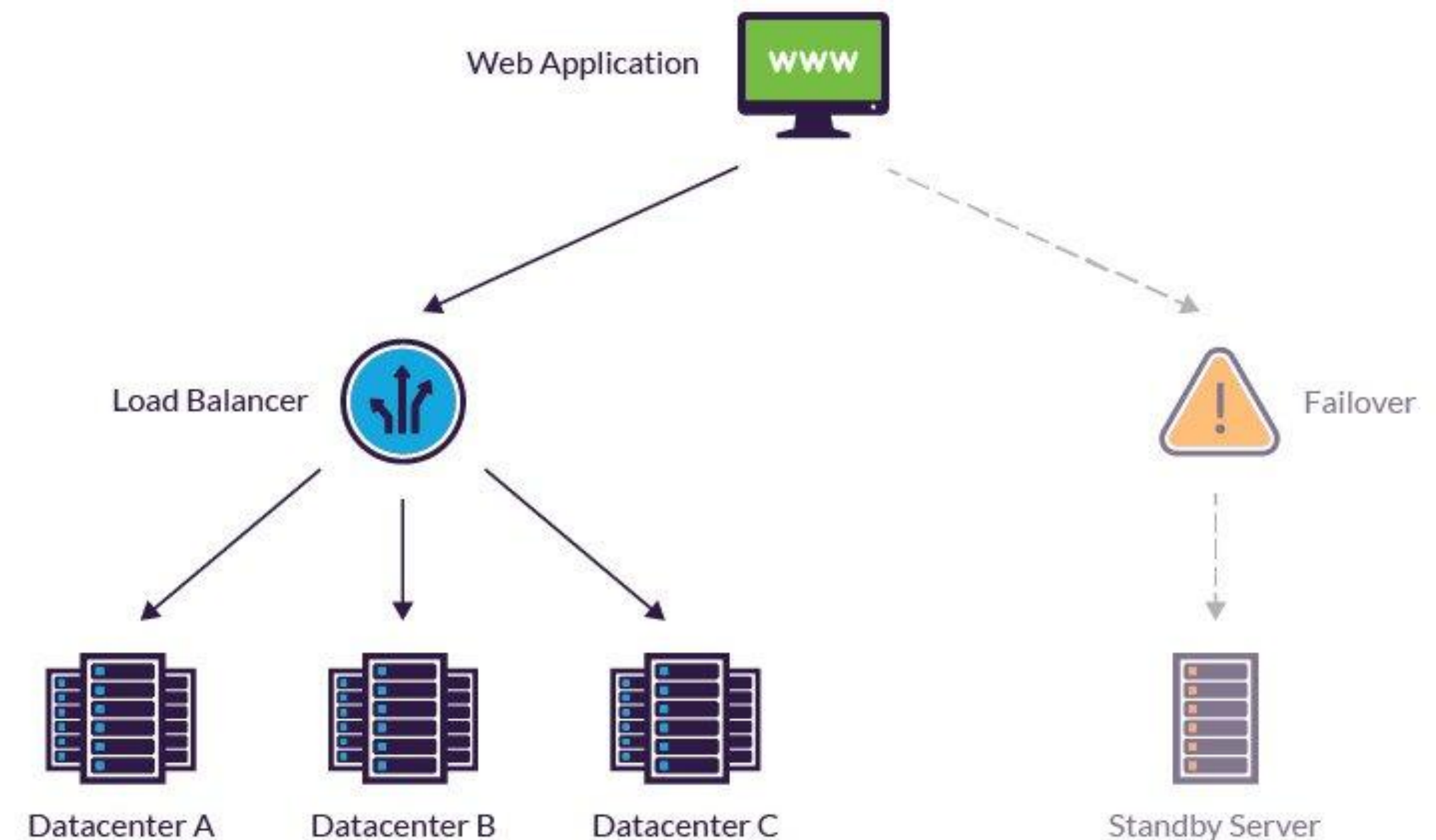
- VM crashes
- hardware/network failure
- zone outage

Fault tolerance: the system continues operating, or recovers quickly, when part of it fails

Some capabilities are provided by the cloud platform; others must be designed into the application

Common strategies:

- Redundant instances
- Failover
- Automatic replacement/restart
- Distribute resources across zones



Migration

Why migrate?

- Hardware or infrastructure maintenance
- Availability or capacity needs
- Move to a different region/service
- Cost or performance changes
- Application modernization

E.g., Compute Engine can live-migrate many VMs during host maintenance.

Two general approaches

- Live (hot) migration — workload continues running with little or no interruption
- Cold migration — workload is stopped before it is moved

Monitoring: Knowing What Is Happening

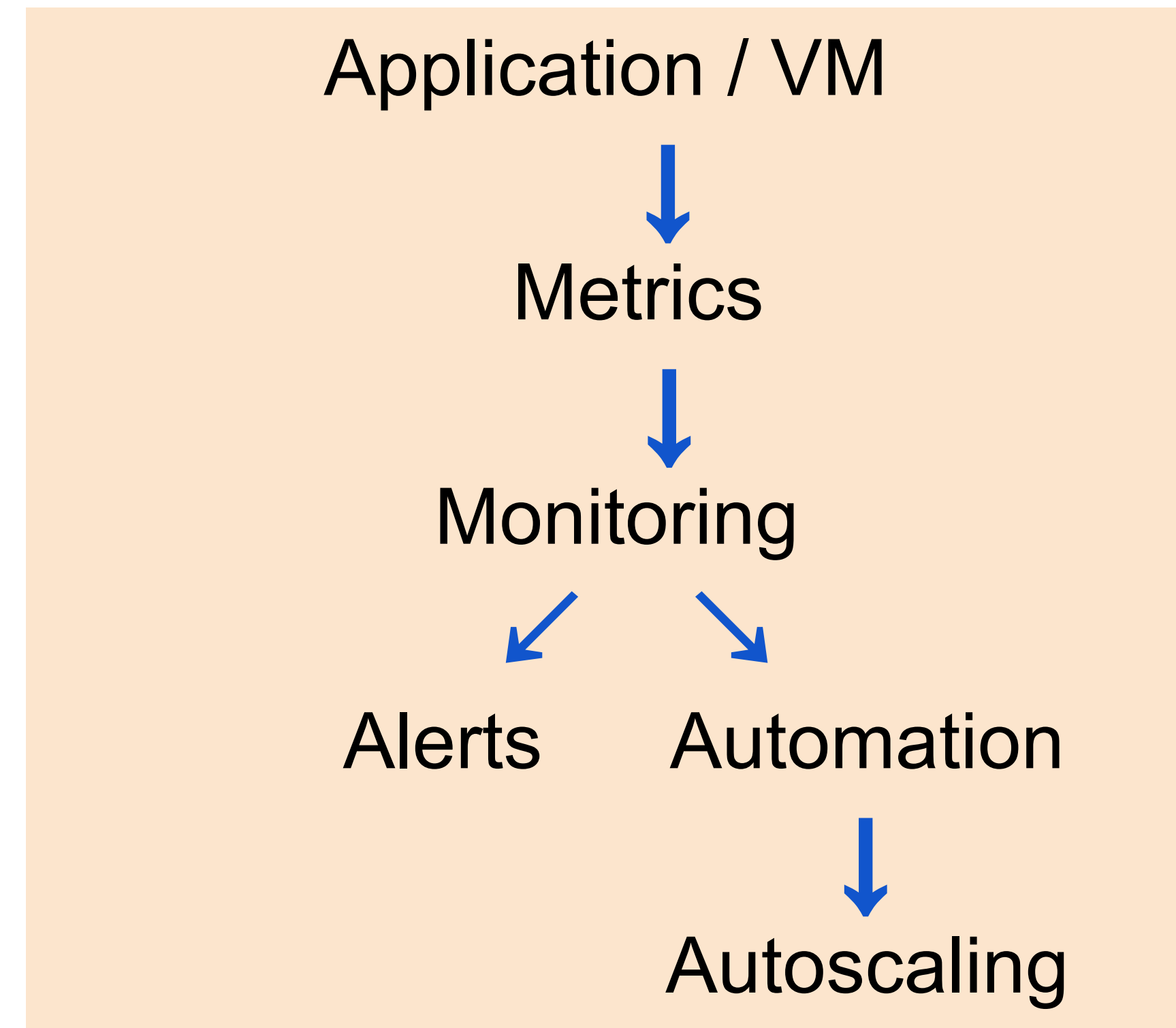
Monitoring: continuously observing the health and behavior of applications and infrastructure

Common things to monitor:

- CPU / memory usage
- network traffic
- response time
- errors / failures
- availability

Monitoring helps us:

- **Alert** people when something is wrong
- **Troubleshoot** problems using metrics and logs
- **Trigger automated recovery** in some architectures
- **Drive autoscaling** based on workload



[Cloud console](#)

Elasticity

Scalability

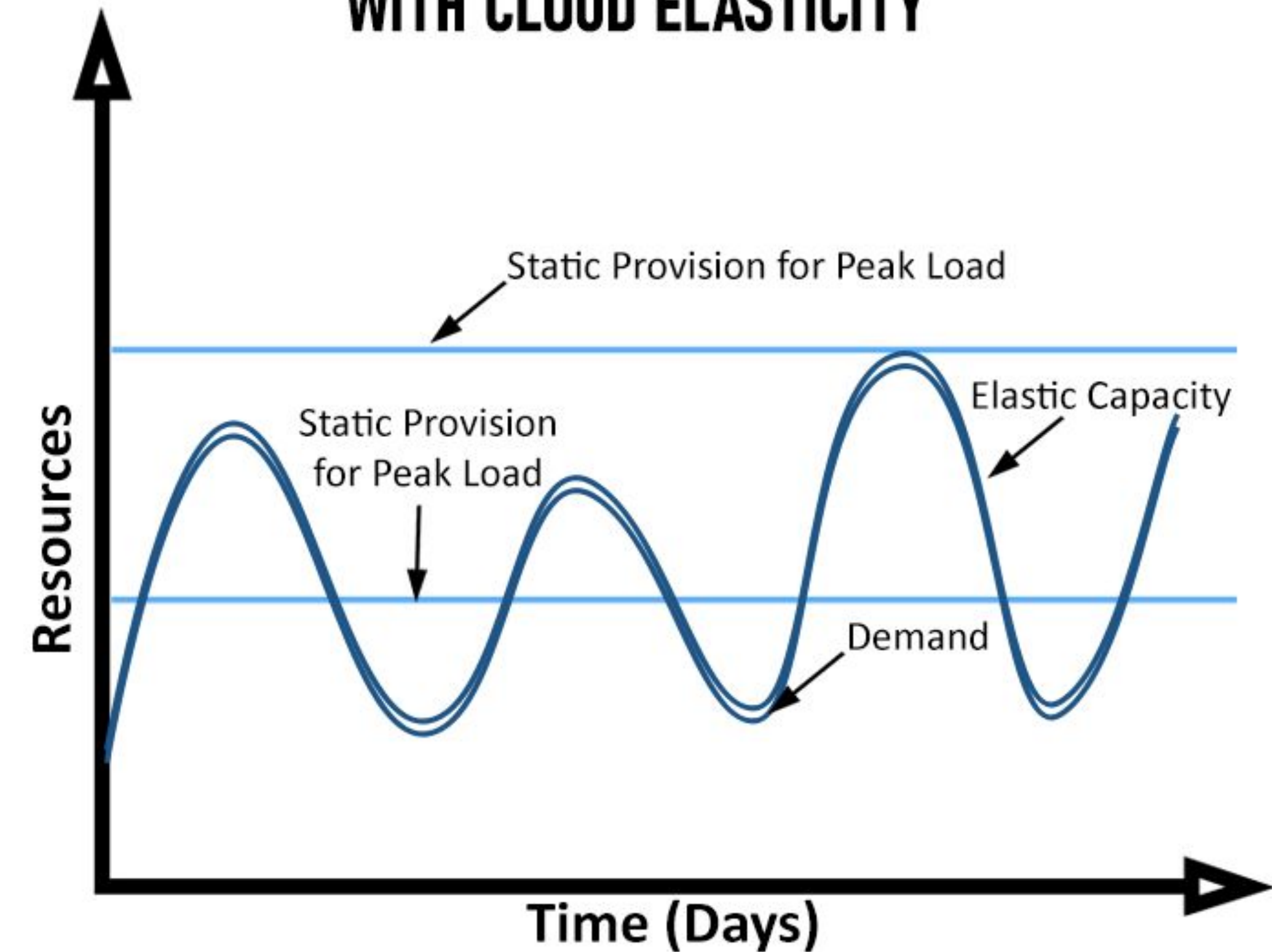
- Ability to handle increased workload by adding resources

Elasticity

- Ability to automatically add or remove resources as demand changes
- Scale out when demand increases
- Scale in when demand decreases
- Relies on monitoring + automation



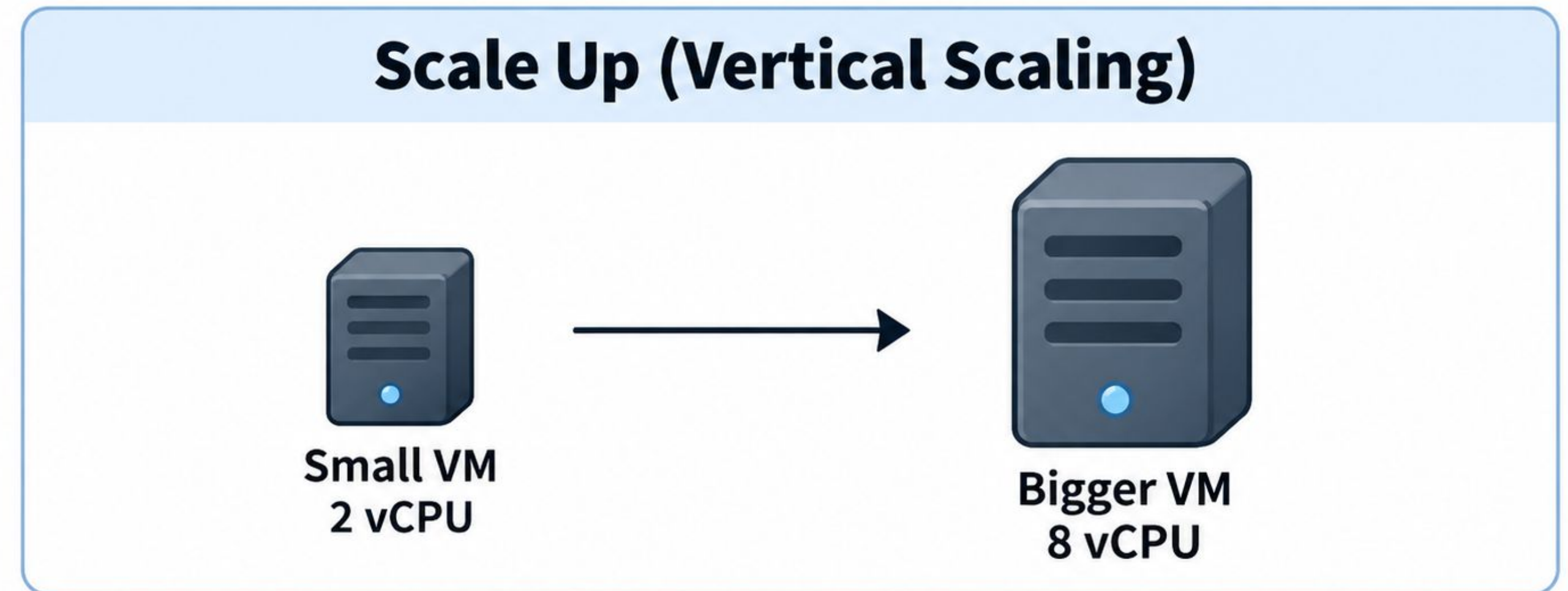
COMPARISON OF STATIC CAPACITY WITH CLOUD ELASTICITY



Scale Up vs Scale Out

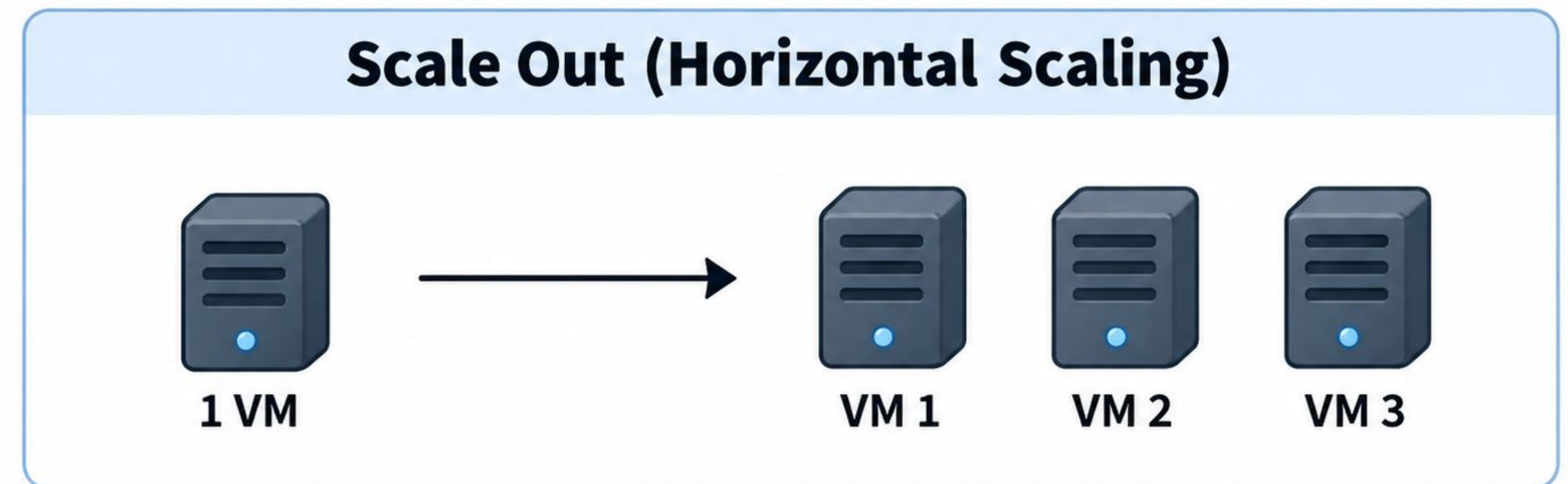
Scale Up (Vertical Scaling)

- Give one machine more resources
- More CPU, memory, etc.
- Example: **e2-micro** → **e2-standard-4**



Scale Out (Horizontal Scaling)

- Add more machines/instances
- Distribute workload across them
- Example: **1 VM** → **4 VMs**



Cloud autoscaling usually focuses on scaling out and scaling in.

Autoscaling in Compute Engine

Managed Instance Group (MIG)

- A group of VMs managed as one unit
- VMs are created from a common instance template

Autoscaler

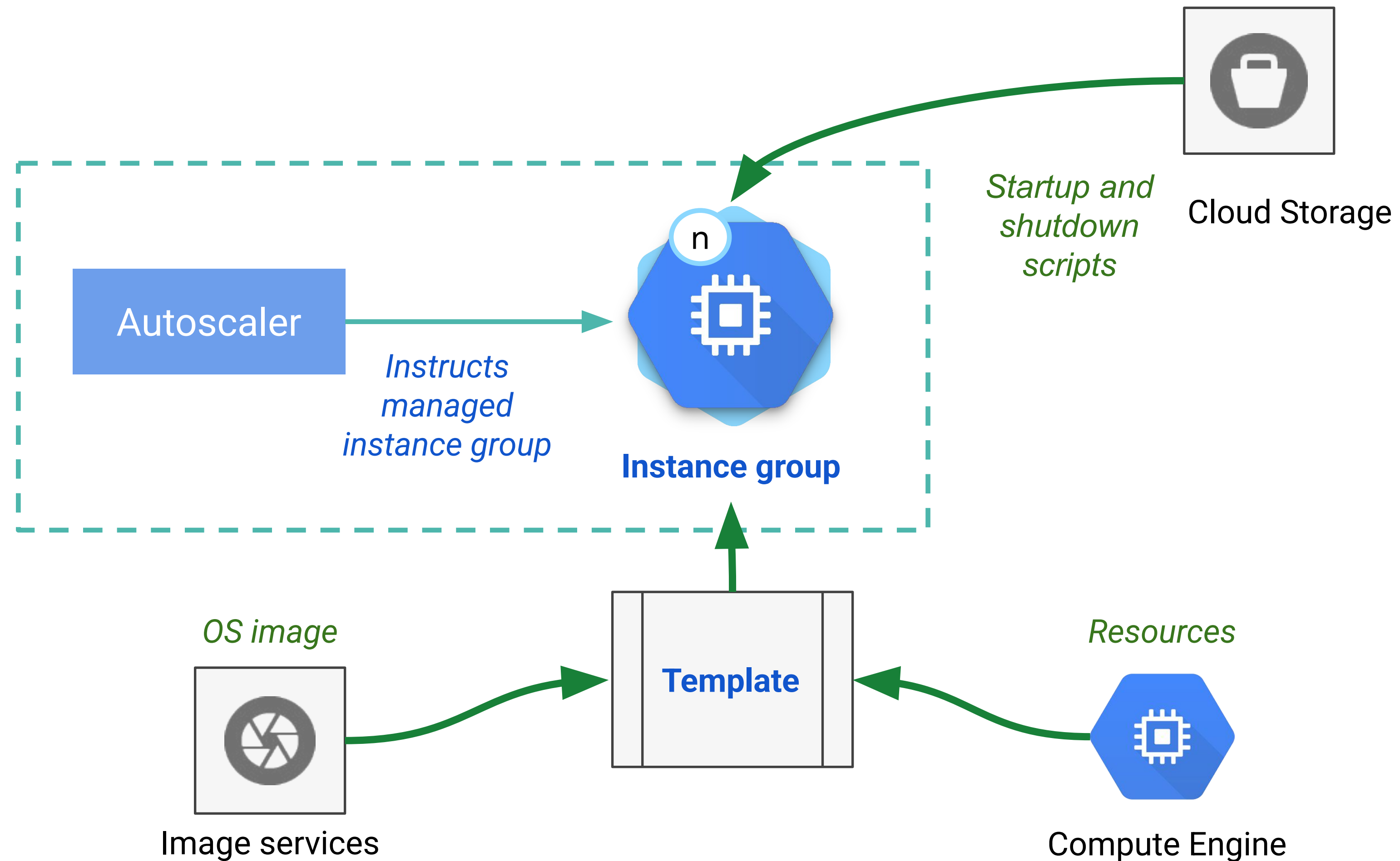
- Monitors workload
- Scale out: add VMs when demand increases
- Scale in: remove VMs when demand decreases

Possible scaling signals

- CPU utilization
- Load-balancing capacity
- Cloud Monitoring metrics
- Schedule

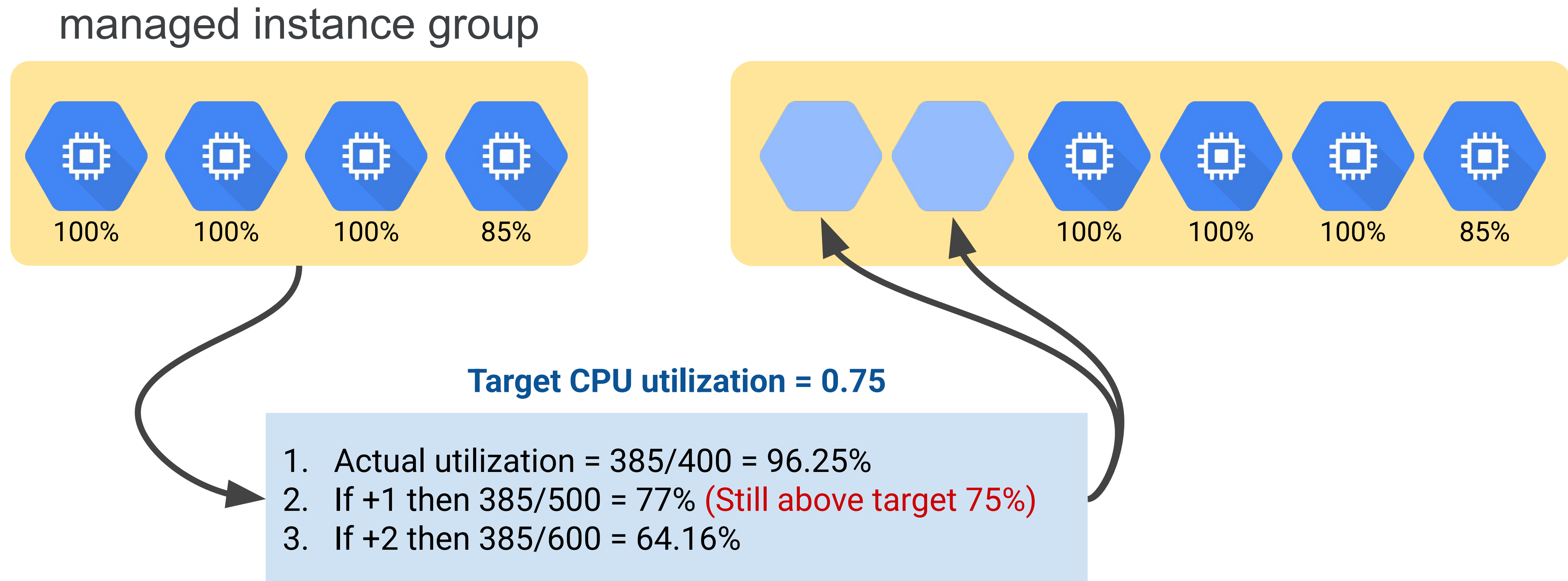
Official Doc: [Instance groups](#)

How autoscaling works

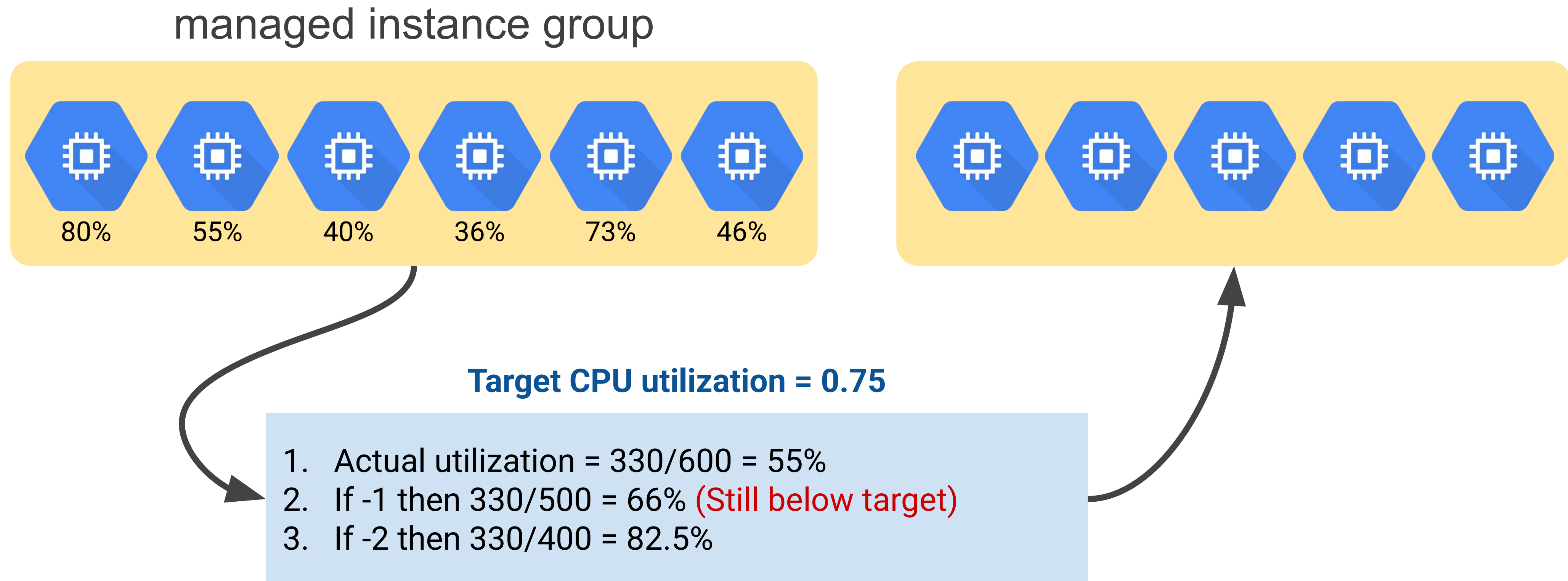


Copyright Google LLC. For educational purposes in accordance with the terms of use set forth on the program [Website](#).

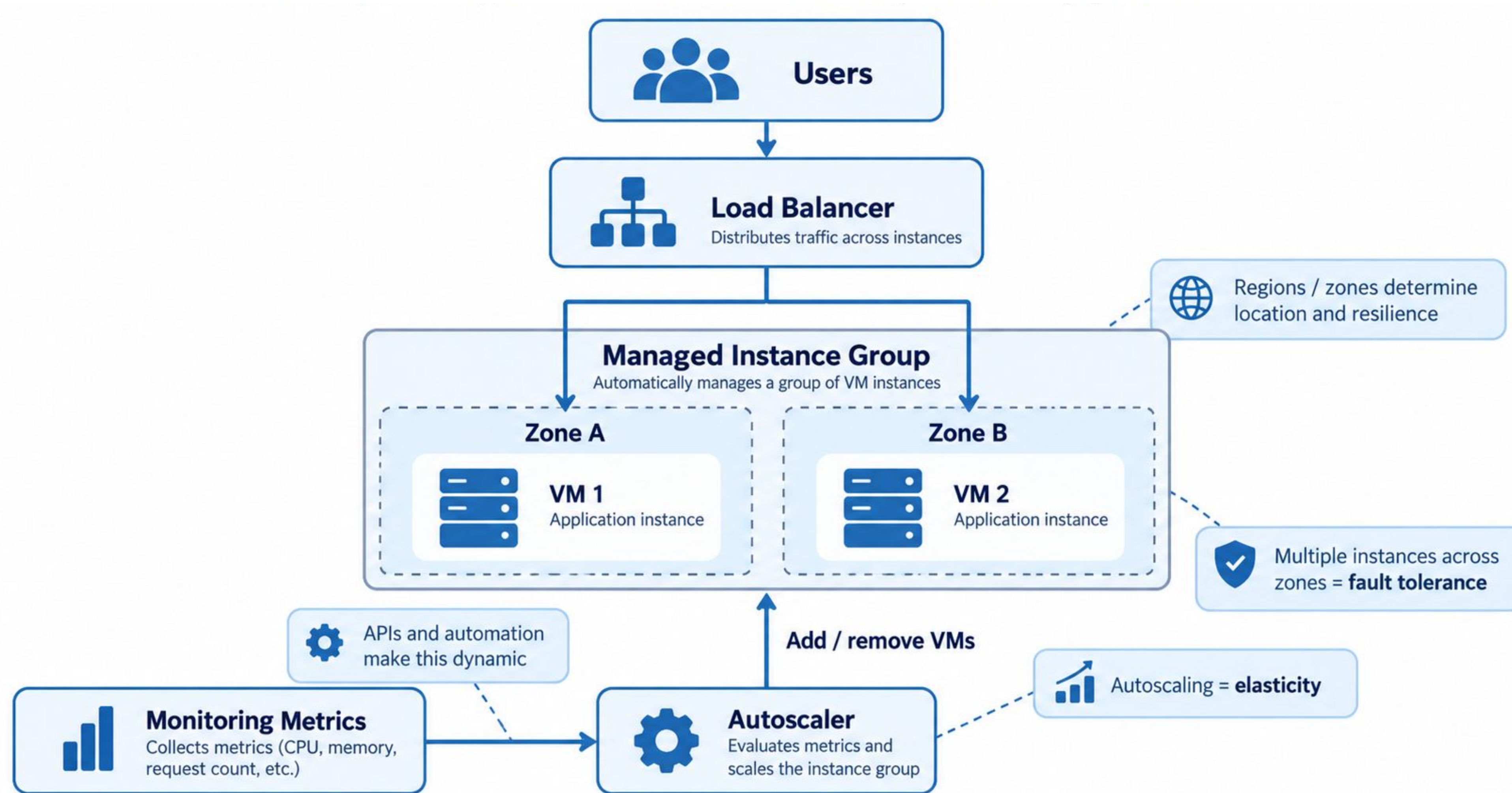
Scale-out policy decision



Scale-in policy decision



Putting It Together: A Cloud-Ready Web Application



Summary

- Cloud applications should be designed to handle change and failure
- APIs enable abstraction and automation
- Regions and zones affect latency and availability
- Fault tolerance uses redundancy, failover, and recovery
- Monitoring provides visibility into system health and workload
- Scalability adds capacity; elasticity adjusts capacity dynamically
- Managed Instance Groups + autoscaling can add/remove VM instances based on demand

Running in the cloud is not the same as being designed for the cloud.

Practice: Design for Changing Demand

Scenario: A university course-registration website normally has moderate traffic. When registration opens at 7:00 a.m., traffic may increase **5–10× within minutes**. The service should also remain available if one VM fails.

- Where would you place the application resources—one zone or multiple zones?
- How would you reduce the impact of a VM failure?
- What metrics would you monitor?
- Would you scale up or scale out when traffic suddenly increases, and how could autoscaling help?

